Reviewer Pack — Minimal Order of Sheaves over Affine Schemes and SAT Clause …

Entry 30942ebc1f58 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 04:39:31 UTC

Minimal Order of Sheaves over Affine Schemes and SAT Clause Subset Entropy Correlation

Entry ID: `30942ebc1f58` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 04:39:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Sheaf Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

> For every SAT clause set φ, the minimal order of sheaves over an affine scheme associated with φ is linearly correlated with the entropy of φ's subset, such that min_order(φ) = Θ(EntSubset(φ)).

Rationale (proposer's reasoning):

> Sheaf theory provides a geometric framework that can potentially capture the structural properties of SAT clauses. The minimal order of sheaves might reflect the complexity or depth of interactions between clauses, which is closely related to their entropy.

Taxonomy category: `Sheaf_Theoretic_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8a5c5cbb7a68fd4a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Spearman's rank correlation coefficient between min_order(φ) and EntSubset(φ) across 30 random seeds is greater than or equal to 0.7, otherwise it is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - min_order sheaf theory AND affine scheme Correlation with SAT clause subsets - entropy EntSubset and minimal order sheaves algebraic geometry - SAT clause subset entropy correlation with sheaf theory minimal order

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_sat_instance(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), random.randint
        clauses.append(clause)
    return clauses

def compute_min_order(clauses):
    n = len(clauses[0])
    min_order = 0
    for i in range(n):
        if all(any(j != k and abs(clause[i]) == abs(clause[k]) for clause in clauses) for j in ran
            min_order += 1
    return min_order

def compute_entropy(subset):
    counts = [0] * len(subset)
    for clause in subset:
        for var in clause:
            if var > 0:
                counts[var - 1] += 1
            else:
                counts[-var - 1] -= 1
    total = sum(abs(count) for count in counts)
    entropy = 0.0
    for count in counts:
        if count != 0:
            p = Fraction(abs(count), total)
            entropy -= p * math.log2(p)
    return entropy
```

```python
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    min_order_sum = 0.0
    entropy_sum = 0.0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            clauses = generate_sat_instance(n, random.randint(1, n * (n + 1) // 2))
            subset_size = random.randint(1, len(clauses))
            subset = random.sample(clauses, subset_size)

            min_order = compute_min_order(subset)
            entropy = compute_entropy(subset)

            min_order_sum += min_order
            entropy_sum += entropy
            instances_tested += 1

    mean_min_order = min_order_sum / instances_tested
    mean_entropy = entropy_sum / instances_tested
    correlation_coefficient = (instances_tested * sum(min_order * entropy for min_order, entropy i
                               mean_min_order * mean_entropy) / math.sqrt((instances_tested * sum(
                                                                   (instances_tested * sum(er

    conjecture_holds = correlation_coefficient >= 0.7
    counterexample = "" if conjecture_holds else "Spearman's rank correlation coefficient < 0.7"

    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif sum(1 for result in results if not result["conjecture_holds"]) / len(results) <= 0.2:
```

```
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68fe4901.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68fe4901.py", line 63, in run_trial
    entropy = compute_entropy(subset)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68fe4901.py", line 40, in compute_entro
    counts[-var - 1] -= 1
    ~~~~~~^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the Spearman's rank correlation coefficient could not be computed. | next: Re-run the test to ensure it completes successfully and produces the required data for correlation analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13507
2	preregistration	ollama_remote	glm4:latest	0	0	9125
3	novelty	ollama_remote	glm4:latest	0	0	8331
4	novelty	ollama_remote	glm4:latest	0	0	9184
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21666
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11804
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13588
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12106
9	judge	ollama_remote	glm4:latest	0	0	18069

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 117378 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/30942ebc1f58.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/30942ebc1f58.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/30942ebc1f58.tar.gz` (if generated)